



Barcelona School of Economics

Before the Buzzer: A Transformer Approach to Early Prediction and Transfer Learning in VR Cognitive Training

Olsa Berani, Jillian Hunter, and Olta Reçica

Barcelona School of Economics

June, 2026

Abstract

Predicting how a VR cognitive training session will end before it is over would give platforms like Virtuleap’s Enhance VR a real window to intervene: flagging struggling users, triggering smarter support, or delivering feedback while it can still make a difference. We build a sequence model (a SessionTransformer) that reads the trial-by-trial behavioral data from the opening of a session and predicts the final session score. Trained on Whack-a-Mole, the most data-rich game in the Enhance VR library, the model explains 78.2% of score variance from 40 trials, a significant improvement over a simpler statistical baseline ($R^2 = 0.782$ vs 0.701 , $p < 0.001$), and retains over 50% explanatory power from just 5 trials. We transfer the pretrained model to seven other games via fine-tuning and find it can outperform the baseline trained on the same limited data, although the strongest results require 200 to 500 labeled sessions. An attribution analysis using Integrated Gradients (a method that traces which inputs most influence the model’s output) identifies normalized reaction time and consecutive-trial streak as the two leading predictive signals across all eight games. These findings show that interpretable early prediction is feasible and that a model pretrained on a data-rich game can outperform a task-specific baseline from as few as 20 labeled sessions in the target game.

Contents

1	Introduction	4
2	Literature review	5
2.1	Brain training and cognitive assessment in virtual reality	5
2.2	Behavioral signals and performance prediction	5
2.3	Knowledge tracing: from recurrent networks to transformers	6
2.4	Transfer learning and few-shot adaptation	6
3	Data	7
3.1	Dataset and problem formulation	7
3.1.1	The platform and dataset	7
3.1.2	Problem statement	8
3.1.3	The K cutoff	9
3.1.4	Train, validation, and test split	9
3.2	Data quality and preprocessing	10
3.2.1	Filtering pipeline	10
3.2.2	Early window coverage	11
4	Methodology	11
4.1	Feature engineering	11
4.1.1	Raw event data	11
4.1.2	The six behavioral features	12
4.2	The SessionTransformer	13
4.2.1	Why a transformer	13
4.2.2	Architecture	13
4.2.3	Training	14
4.3	Transfer learning across games	15
4.3.1	Motivation	15
4.3.2	Fine-tuning protocol	15
4.3.3	Baselines for transfer	15
4.4	Interpretability: Integrated Gradients	16
4.4.1	Why Integrated Gradients	16
4.4.2	Method	16
5	Results and discussion	17
5.1	Early session score prediction	17
5.1.1	Performance across trial cutoffs	17
5.1.2	The transformer outperforms Ridge at all K	18
5.1.3	Training dynamics	19
5.2	Transfer learning across games	20
5.2.1	Zero-shot transfer fails	20

5.2.2	Fine-tuning with limited target data	20
5.2.3	Transfer performance varies by game	21
5.2.4	The Shuffled case	21
5.3	Interpretability	22
5.3.1	Reaction time and streak are leading predictors across games	22
5.3.2	Direct accuracy measures are weaker than expected	23
5.3.3	Temporal attribution patterns	23
5.3.4	Streak vs reaction time through time	24
5.4	Synthesis	25
6	Limitations and future work	25
6.1	Limitations	25
6.2	Future work	26
7	Conclusion	27
	Appendices	29
A	Dataset summary	29
B	Transfer learning results	29
C	Pairwise feature correlations	30
D	WaM performance across sessions	31
E	Ridge regression baseline	31
F	Model architecture and hyperparameters	32

1 Introduction

Predicting how a session will end matters most while there is still time to change its course. At Virtuleap’s Enhance VR, a VR-based cognitive training platform with around 70,000 registered users, the data generated during each session create an opportunity to make that prediction early (Saredakis et al., 2022). Users wear a headset and play short gamified sessions targeting different cognitive functions, ranging from memory and attention to processing speed and problem-solving. Each game session is composed of individual trials, and information regarding each trial is recorded: whether the user succeeded, how quickly they responded, and any game-specific outputs. At the end of each session, the platform assigns the user a score between 0 and 100 based on their performance. Virtuleap already uses this data to adjust difficulty in real time, but the behavioral signal each session produces goes further than difficulty adaptation. Beyond adapting to the user’s current level, Virtuleap’s longer-term goal is to build meaningful cognitive reports that track how users are developing over time. This goal depends on understanding the behavioral patterns contained in each session. The question this thesis addresses is whether that understanding can start within a session.

A session score tells you what a user achieved, but two users with the same score may have gotten there in completely different ways, one recovering steadily after early mistakes, another starting strong and fading. The sequence of individual trials within a session carries that structure, and a model that reads the sequence can pick it up in ways that aggregate statistics cannot. If a reliable estimate of the final score can be generated from just the first few trials, the platform gains a window to act: flag users at risk of dropping off, trigger smarter difficulty adjustments earlier in the session than the current system allows, or deliver personalized feedback before a session goes wrong. This is particularly valuable for Virtuleap because the platform serves a broad range of users, from those who are cognitively healthy and training for performance, to those who may be in early stages of cognitive decline. Catching a struggling session early could also provide a valuable signal for richer longitudinal cognitive reporting.

Work in educational technology on knowledge tracing, estimating how a student is doing from a sequence of exercise responses, points toward the right kind of model for this. Deep Knowledge Tracing (Piech et al., 2015) showed that sequence models trained on trial-by-trial outcomes consistently outperform approaches that only look at summary statistics, and time-aware transformer architectures like SAINT+ (Shin et al., 2021) have extended this further. Our work applies the same idea to VR cognitive games, where the prediction target is a continuous session score rather than a binary correct-or-incorrect response. With 17 games in the library, training a separate model for each one from scratch requires more labeled data than is available for newer or less popular titles. This is the cold start problem: the moment when a model would be most useful is exactly when it has the least data to work with. Few-shot fine-tuning offers a way around this, with pretrained models adapting to new domains from very limited examples (Zhao et al., 2023).

This thesis makes three contributions. First, we train a SessionTransformer on WaM trial sequences and establish that within-session score prediction is feasible from as few as five early trials. Second, we test whether the pretrained model transfers to seven other games via few-shot fine-tuning, comparing it against Ridge regression at matched data budgets and on the full training set. Third, we apply Integrated Gradients (Sundararajan et al., 2017) to identify which behavioral features drive predictions across all eight games.

2 Literature review

2.1 Brain training and cognitive assessment in virtual reality

Digital cognitive training has grown substantially as both a research tool and a consumer product, driven by the promise that structured mental exercise can improve performance across cognitive domains. The evidence is more mixed than the industry has suggested. Studies on computerized working memory training consistently find improvements on the trained tasks, but gains on untrained tasks are small and the evidence for transfer to everyday cognition is weak (Li et al., 2026; Simons et al., 2019). The commercial brain training industry has not always been transparent about these limits. Lumosity was fined \$2 million by the FTC in 2016 for overstating the cognitive benefits of its platform (Federal Trade Commission, 2016), which points to a genuine need for more rigorous tools for understanding what users actually learn during training.

Virtual reality offers a meaningful advance over screen-based approaches in this respect. By delivering cognitive exercises within immersive, naturalistic environments, VR systems can combine the ecological validity of real-world tasks with the experimental control of a laboratory setting. Enhance VR, developed by Virtuleap, is one of the more carefully designed platforms in this space (Saredakis et al., 2022). It delivers short gamified exercises across seven cognitive categories, adapts difficulty dynamically to each user’s performance, and collects structured trial-by-trial behavioral data, including reaction times and motor outputs, across every session. Virtuleap frames this data as a potential source of non-invasive digital biomarkers of cognitive status. The question our work addresses is how to extract predictive and interpretable signal from it.

2.2 Behavioral signals and performance prediction

Session scores summarize what a user achieved but say little about how they got there. The trajectory of performance within a session, how quickly a user responds, whether they recover after errors, how accuracy evolves over time, encodes structure that aggregate metrics discard. A line of work in VR and gamified learning has begun to show that this behavioral signal is genuinely predictive. Studies using VR interaction data have demonstrated that in-session behavioral features can classify users by skill level with meaningful accuracy (Wismer et al., 2022), and related approaches have used early session performance metrics to forecast whether users will continue engaging with a gamified

cognitive training program (Pang et al., 2024). Both treat in-session behavior as a source of prediction. The key limitation shared by this work is reliance on aggregate features, summaries of behavior across a session rather than the sequence of individual trials. The ordering of events within a session, which carries information about momentum, recovery, and consistency, is lost in aggregation.

Early prediction sharpens the practical value of this approach further. If a model can estimate where a user is headed from the first few trials of a session, that estimate can inform real-time decisions about training structure and content delivery. Even on a platform that already adapts difficulty dynamically, prediction at the trial-sequence level works at a finer granularity than difficulty adjustment alone, and opens the possibility of richer cognitive reporting.

2.3 Knowledge tracing: from recurrent networks to transformers

The most directly relevant technical literature comes from knowledge tracing, a subfield of educational technology concerned with estimating a learner’s underlying knowledge state from sequences of exercise responses. The core problem is similar in structure to ours: given a series of trial-level outcomes, predict future performance.

Deep Knowledge Tracing (Piech et al., 2015) established the modern framing of this problem by replacing earlier probabilistic models with an LSTM trained on sequences of student interactions, producing substantial gains over Bayesian methods and demonstrating that neural sequence models could extract meaningful signal from both the ordering and frequency of responses. Self-attentive models such as SAKT (Pandey and Karypis, 2019) later showed that attention-based architectures outperform recurrent ones on the same task, and SAINT+ (Shin et al., 2021) extended this further by incorporating elapsed time and lag time as explicit features, achieving strong results on the EdNet dataset and demonstrating the value of timing information in student response sequences.

Our SessionTransformer follows this lineage. It applies the same self-attentive structure to trial sequences from VR cognitive games, replacing the binary correct-or-incorrect response with a richer six-feature representation per trial and shifting the prediction target from next-item correctness to continuous session score. The underlying structure is the same: a user producing a sequence of responses whose trajectory reflects their current performance level. What differs is that our setting demands early prediction of a session outcome rather than forecasting the next item, and the behavioral signal available per trial is richer than a single binary outcome.

2.4 Transfer learning and few-shot adaptation

Training a separate model for each game from scratch is impractical across the eight games in this dataset, where labeled data per game varies considerably. Transfer learning addresses this by initializing a model with representations learned in a data-rich source domain before fine-tuning on the target. The question is how much target data is needed for the result to be useful.

Previous work on source-free domain adaptation shows that carefully fine-tuned pre-trained models can adapt using very few labeled target examples without severe overfitting on the evaluated benchmarks (Zhao et al., 2023). Evidence from cross-domain few-shot learning also suggests that transfer depends partly on the structure shared by the source and target tasks (Lin et al., 2021). Our experiments extend this question to VR cognitive training by testing whether representations learned from WaM remain useful across games with different mechanics and sequence lengths. The variation in performance provides a starting point for studying which forms of similarity matter most for transfer.

3 Data

3.1 Dataset and problem formulation

3.1.1 The platform and dataset

Enhance VR, developed by Virtuleap, is a VR-based cognitive training platform that delivers short gamified exercises targeting seven distinct cognitive domains: memory, attention, processing speed, motor control, orientation, task flexibility, and problem-solving (Saredakis et al., 2022). Users wear a VR headset and complete short game sessions, each made up of a series of trials that require quick reactions and accurate responses. At the end of each session, the platform computes a summary score between 0 and 100 that reflects overall accuracy and speed relative to the difficulty level the user encountered. Figure 1 shows this structure.

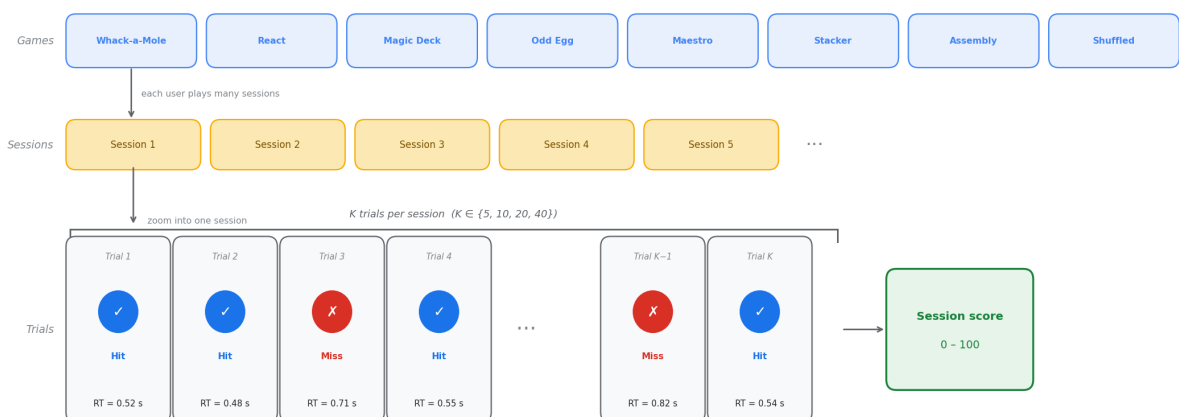


Figure 1: Structure of the dataset. Each of the eight games contains many sessions per user. Each session is a sequence of K trials, where every trial records whether the user hit or missed and their reaction time. The session ends with a summary score between 0 and 100.

The Enhance VR platform includes 17 games in total; the dataset provided for this project covers eight of them: Whack-a-Mole (WaM), React, Assembly, Maestro, Magic

Deck, Odd Egg, Stacker, and Shuffled. The session metadata and per-game event files are matched and filtered as described in Section 3.2, yielding 109,915 usable sessions across the eight games. We build our base model on WaM because it is the most data-rich game in the set with a clean, accuracy-based trial structure that is well-suited for learning behavioral representations from sequences; it contributes 23,345 usable sessions from 4,932 users with a score distribution of 54.8 ± 31.1 . Figure 2 shows the distribution of session scores across all eight games.

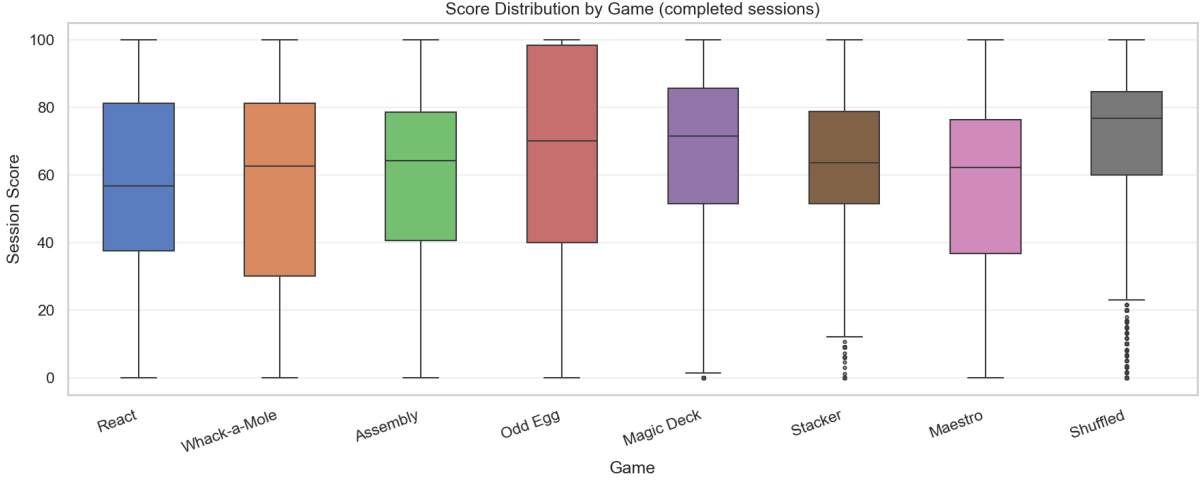


Figure 2: Distribution of session scores across all eight games. Scores are bounded between 0 and 100. WaM shows a wide spread centered around 54.8, while other games vary considerably in their score distributions.

3.1.2 Problem statement

The core task is to predict a user’s final session score from only the first K trials of that session. Formally, given a sequence of trial feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_K \in \mathbb{R}^6$, we want to learn a function f such that $f(\mathbf{x}_1, \dots, \mathbf{x}_K) \approx y$, where $y \in [0, 100]$ is the session score. This predictive task estimates where a session is heading and does not support causal conclusions about what makes a user perform well or poorly.

The motivation for framing this as an early prediction problem, rather than a full-session regression, is practical. A score computed at the end of a game is informative but arrives too late to act on. If a reliable estimate can be generated from the opening trials alone, the platform gains a meaningful window in which to adjust difficulty in real time, flag sessions that show signs of performance decline, or trigger a support mechanism before a user disengages entirely.

We address this problem at two levels. First, we train and evaluate a model on WaM to establish whether early prediction is feasible. Second, we test whether the learned representations transfer to the remaining seven games via fine-tuning, asking how little per-game data is needed to achieve competitive performance. This second question has direct practical relevance: a system that generalizes across games with minimal labeled data can be deployed platform-wide without requiring a separate data collection effort for

each new game.

3.1.3 The K cutoff

As a result of game sessions varying in total trial count across the library, we define K per game as a fixed number of early trials. Across the eight games this ranges from 5 trials for short-window games such as Shuffled and Odd Egg, up to 40 for WaM and React. Table 1 shows the K value and the raw event column mappings for each game. Section 3.2 shows what fraction of a typical session each cutoff covers. We run a K -sweep on WaM, testing $K \in \{5, 10, 20, 40\}$, to compare predictive performance at different operational cutoffs. Each cutoff retains all sessions with at least K trials, so the eligible cohorts differ slightly: the test set contains 2,917 sessions at $K = 5$ and 2,797 at $K = 40$. The sweep therefore describes performance at each cutoff rather than a controlled estimate of the effect of adding trials. The results are reported in Section 5.1.

Game	K	Hit column	RT column
WaM (base)	40	outcome (hit mole)	time to hit [s]
React	40	successful action	action time (derived)
Assembly	13	success	reaction time (s)
Maestro	12	was response correct	response time [s]
Magic Deck	6	correct Selection	time to select (sec)
Odd Egg	5	is selected odd	time to deliver (sec)
Stacker	6	optimal move	time to move [s]
Shuffled	5	success	time to answer (sec)

Table 1: Games, trial cutoffs (K), and the column names used to extract hit outcome and reaction time from the raw event tables.

3.1.4 Train, validation, and test split

We split the data at the user level, assigning 70% of users to training, 15% to validation, and 15% to test, with a fixed random seed for reproducibility. For WaM, this produces 3,452 training users (17,457 sessions), 740 validation users (2,970 sessions), and 740 test users (2,918 sessions).

The decision to split by user rather than by session is important for evaluation validity. A session-level split would allow the model to see earlier sessions from a given user during training and then be evaluated on that same user’s later sessions, which introduces a form of leakage: the model can implicitly learn about individual users rather than generalizing to new ones. A user-level split avoids this by ensuring the test set contains only users the model has never encountered during training. This is also the correct framing for deployment, where the platform would apply the model to new users whose behavioral histories are completely unseen.

3.2 Data quality and preprocessing

3.2.1 Filtering pipeline

The raw session log contains records for all 17 games across the platform’s lifetime. Before analysis, we apply three sequential filters. First, we restrict sessions to `status_id = 2` (completed) and `is_tutorial = False`, yielding 142,751 sessions from 11,304 users across the eight games in scope, collected between August 2020 and March 2026. Second, we match each session to its event records by session ID and drop sessions absent from the corresponding event file or with a missing score value. Third, we exclude sessions with fewer than K trial records.

Table 2 shows session counts at each stage by game.

Game	Completed sessions	Usable sessions	Dropped (%)
WaM	25,764	23,345	9.4
React	33,456	9,986	70.2
Assembly	17,896	16,225	9.3
Maestro	12,649	12,129	4.1
Magic Deck	14,927	13,233	11.3
Odd Egg	16,839	15,176	9.9
Stacker	12,715	11,580	8.9
Shuffled	8,505	8,241	3.1
Total	142,751	109,915	23.0

Table 2: Session counts before and after applying the event-matching and K -trial-minimum filters. “Completed sessions” is the count of non-tutorial sessions with `status_id = 2` across the eight games. “Usable sessions” are those successfully matched to event records with a valid score and at least K trials.

React loses 70.2% of its completed sessions, the largest drop of any game. The primary cause is missing event records: many React sessions recorded as completed in the session log have no corresponding rows in the event file, likely because detailed trial-level logging was not yet active for this game during an earlier period of platform deployment. For all other games, attrition is below 12%, with the remaining loss coming mainly from sessions that fell short of the K -trial minimum. The usable React cohort therefore covers only sessions with both a completion record and a full event log, which is worth noting when interpreting the transfer results in Section 4.3. React also differs in how reaction time is recorded: the event file stores action timestamps rather than elapsed response times, so we reconstruct reaction times by computing successive timestamp differences within each session, assigning the first trial a value of zero.

3.2.2 Early window coverage

We chose the K values in Table 1 to represent a practically meaningful early window. Figure 3 places each K against the median total trial count per session, showing what fraction of a typical session the cutoff covers.

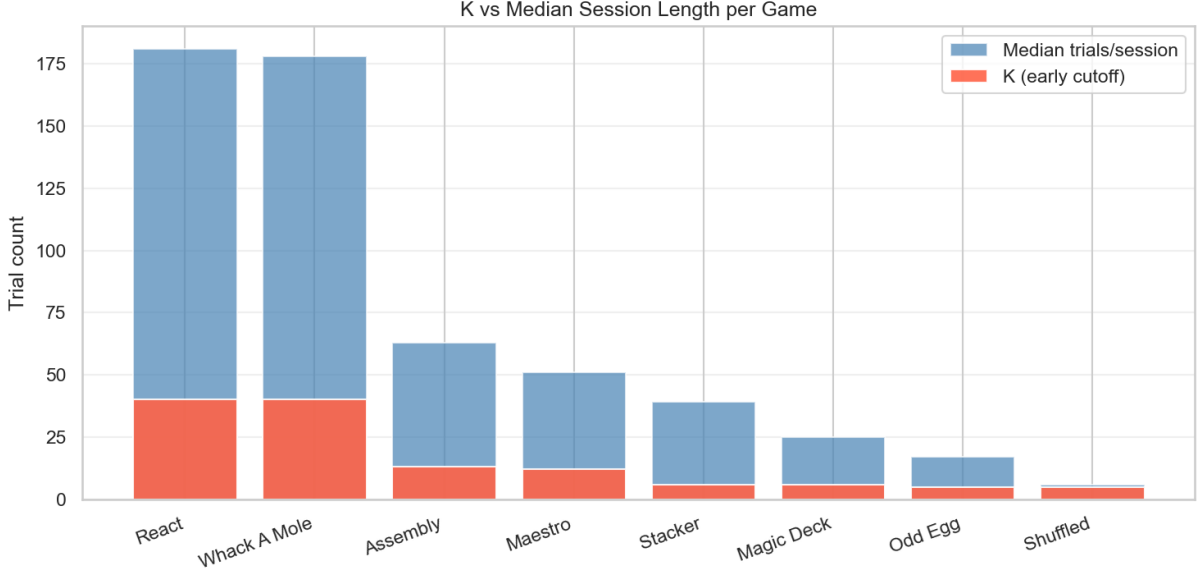


Figure 3: K (red bar) plotted against the median total trial count per session (blue bar) for each game. For all games except Shuffled, K covers between 15% and 30% of a median session.

For WaM and React, $K = 40$ covers approximately 22% of a median session of 178–181 trials. Assembly (median 63 trials, 21%), Maestro (median 51, 24%), and Magic Deck (median 25, 24%) are comparable. Stacker ($K = 6$, median 39 trials) and Odd Egg ($K = 5$, median 17 trials) cover 15% and 29% respectively.

Shuffled is an exception. Its median session contains only six trials, so $K = 5$ covers 83% of a typical session rather than a genuine early window. With so few trials available, there is little behavioral trajectory to observe before the session ends. The weak predictive performance for Shuffled reported in Section 5.2 follows directly from the limited information available in the first five trials.

4 Methodology

4.1 Feature engineering

4.1.1 Raw event data

Each game’s event table has one row per trial, with the hit outcome column and reaction time column varying by game as shown in Table 1. The hit outcome is encoded as 1 for a successful trial and 0 otherwise, using the game-specific conditions shown in Table 1.

Reaction times are then normalized in two steps across all games. First, values are capped at the 99th percentile of the training split, removing extreme outliers without discarding the trials themselves. Second, the capped values are z-score normalized using the mean and standard deviation of the training split. Missing reaction time values are filled with the cap before normalization. Both statistics are computed on training data only and applied as fixed constants to the validation and test splits, so that no test-set information enters the input features. For WaM, the resulting constants are: cap at 1.186 s, mean 0.822 s, standard deviation 0.320 s.

4.1.2 The six behavioral features

The goal of feature engineering here is to compress each trial into a compact vector that captures the behavioral dimensions most relevant to predicting the final score, while keeping the feature space small enough to train robustly on the available data. Every trial is represented as a vector of six features covering three distinct dimensions of within-session behavior: accuracy, speed consistency, and momentum.

- **hit:** Whether the player succeeded on this trial (1 or 0). This is the most direct behavioral signal available and forms the backbone of everything else computed from the sequence.
- **rt_norm:** Z-score-normalized reaction time. In timed cognitive tasks, faster responses generally reflect better perceptual processing and motor execution. Normalizing to the training distribution means the feature encodes relative speed rather than absolute speed, making it comparable across users regardless of their individual baseline tempos.
- **running_acc:** The cumulative hit rate from trial 1 up to and including trial t . This gives the model a running summary of how accurately the session has gone so far, without requiring it to hold that information implicitly across all prior time steps.
- **rt_rolling_std_norm:** The standard deviation of reaction time over the five most recent trials, normalized by the global training standard deviation. This captures moment-to-moment consistency in response speed, distinguishing a player who is settling into a reliable rhythm from one whose timing is becoming erratic. Intra-individual variability in reaction time is among the most reliable behavioral signals of cognitive performance, with higher variability linked to early cognitive decline (Jutten et al., 2023).
- **streak:** The length of the current consecutive run of hits (positive values) or misses (negative values). Consecutive success in skilled-performance tasks is a well-documented signal of psychological momentum, associated with improved confidence and better focus (Iso-Ahola and Dotson, 2016). Encoding streak directly gives the model access to this momentum signal without requiring it to infer it from the raw hit sequence.

- **prev_miss:** Whether the immediately preceding trial was a miss (1 or 0). This captures the recency of failure as a separate signal from streak, since a single recent miss after a long hit run carries different information from the same miss embedded in an ongoing losing streak.

Together, these six features form a compact but informative representation of within-session behavioral dynamics. Each encodes information that is not redundant with the others: **hit** gives the moment-level outcome, **running_acc** gives the session-level accuracy trend, **streak** gives the momentum structure, **prev_miss** gives the immediate prior context, **rt_norm** gives the speed picture, and **rt_rolling_std_norm** gives the consistency picture. The design is intentionally simple, leaving the model to learn from the temporal ordering of the features without relying on a large set of precomputed summaries.

4.2 The SessionTransformer

4.2.1 Why a transformer

For trial sequences, the relevance of any given trial depends on the event itself, its position, and the trials surrounding it. Self-attention, the core mechanism of transformer models, allows the model to learn these dependencies directly, without imposing any assumptions about which trials matter most.

The ordering of trials within a session carries predictive information that aggregate approaches discard entirely. A hit on trial 3 followed by a miss on trial 4 is a different event from a miss on trial 3 followed by a hit on trial 4, even though both produce the same per-trial counts. More practically, a player who misses the first five trials and then recovers is in a meaningfully different cognitive state from one who starts strong and fades, and distinguishing these trajectories requires reading the sequence in order.

The results in Section 5.1 show that the SessionTransformer substantially outperforms Ridge regression at every tested cutoff, with the advantage growing as K increases. This supports the hypothesis that the model extracts temporal structure that aggregate approaches cannot capture. The transfer learning results in Section 5.2 provide an additional test of whether the learned representations generalize across seven games with different mechanics and sequence lengths.

4.2.2 Architecture

The model takes a matrix of shape $(K, 6)$ as input and produces a single scalar prediction. The processing pipeline has five stages:

1. **Input projection:** A learned linear layer maps each trial’s 6-dimensional feature vector into a 64-dimensional embedding, giving the model a higher-dimensional space in which to learn trial representations.
2. **Positional encoding:** Sinusoidal positional encodings are added to each trial embedding before it enters the encoder. These encodings give the model explicit

information about the absolute position of each trial within the sequence, which is necessary because self-attention on its own has no sense of order and treats the sequence as a set.

3. **Transformer encoder:** Two stacked encoder layers, each with $d_{\text{model}} = 64$, 8 attention heads, a feedforward sublayer of width 128, and dropout of 0.1. With two layers and a model dimension of 64, the encoder is compact but sufficient to capture relationships between trials at different points in the sequence.
4. **Global mean pooling:** After the encoder, the K trial representations are averaged across the sequence dimension, giving a single 64-dimensional vector that summarizes the whole session. We use mean pooling rather than taking only the final token’s representation because it gives every position an equal initial weight, with the encoder free to learn position-dependent patterns through attention.
5. **Regression head:** The pooled vector passes through a two-layer MLP, $\text{Linear}(64 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(32 \rightarrow 1)$, which projects it to the predicted score.

The total parameter count is 69,505. Given that the WaM training split contains 17,457 sessions, this size is deliberate: large enough to learn meaningful patterns from the sequence, but small enough that the risk of overfitting is well managed by the regularization choices described in the next section.

4.2.3 Training

We train with mean squared error loss, which is the natural objective for a continuous regression target on a bounded scale and penalizes large prediction errors more heavily than mean absolute error. We optimize using the AdamW optimizer (Loshchilov and Hutter, 2019) at a learning rate of 3×10^{-5} and weight decay of 10^{-4} . The learning rate schedule follows a two-phase structure: a linear warmup over the first 10 epochs that gradually increases the learning rate from near zero, followed by cosine annealing for the remaining epochs that smoothly decays it to 10^{-6} . The warmup phase is important for transformer training specifically, as starting with a large learning rate tends to produce large, destabilizing gradient updates before the attention weights have settled into a sensible configuration.

Training runs for up to 500 epochs with a batch size of 128, with gradient norms clipped to 1.0 to prevent occasional large gradient spikes from disrupting training. Early stopping monitors validation loss with a patience of 20 epochs: if no improvement is seen within that window, training halts and the best checkpoint is restored. In practice, the WaM base model at $K = 40$ reaches its best validation loss at epoch 147 and stops at epoch 167. Full architecture and training hyperparameters are listed in Appendix F. With the base model established on WaM, the next question is whether what it learned transfers to the other seven games.

4.3 Transfer learning across games

4.3.1 Motivation

WaM is the most data-rich game in the Enhance VR library, which is why it serves as the source domain for pretraining. But the platform has eight games in this dataset alone, and in a live deployment context each new game starts with zero labeled sessions. The practical question for Virtuleap is whether it needs to collect thousands of sessions per game before a prediction model becomes useful, or whether a model pretrained on WaM can be adapted to a new game quickly. If the encoder has learned general representations of trial-level behavioral dynamics, covering how accuracy trajectories evolve, what momentum patterns look like, and how reaction time variability signals cognitive state, then those representations should transfer across games even when the mechanics differ. Testing this is the goal of the transfer learning experiments.

4.3.2 Fine-tuning protocol

For each of the seven target games, we load the WaM checkpoint and fine-tune the entire model on N sessions sampled randomly from that game’s training split. All parameters are updated during fine-tuning rather than freezing the encoder, which allows the model to adapt its internal representations to the new game’s behavioral structure rather than only adjusting the output layer. We use a learning rate of 10^{-4} for fine-tuning, roughly three times higher than the 3×10^{-5} used during WaM pretraining, to allow faster adaptation given the smaller data budget. All other training settings remain the same: the same warmup and cosine annealing schedule, early stopping with patience 20, and gradient clipping at 1.0.

We sweep the fine-tuning budget $N \in \{0, 10, 20, 40, 50, 100, 200, 500\}$ to understand how performance scales with the amount of target-game data available. The $N = 0$ case corresponds to zero-shot evaluation, where the WaM model is applied directly to the target game without any adaptation. Each positive budget uses one independently sampled set of training sessions with a fixed random seed.

4.3.3 Baselines for transfer

Three baselines allow us to isolate what the pretrained encoder is actually contributing at each N :

- **Zero-shot** ($N = 0$): The WaM model applied directly to the target game without fine-tuning. A positive zero-shot result would mean the pretrained representations already transfer without any adaptation; a negative result means the output head is calibrated to WaM’s score range and does not generalize to the new game’s distribution.
- **Ridge**(N): Ridge regression trained from scratch on the same N sessions as the fine-tuned transformer. Holding the data budget constant provides a useful comparison

with a simple model trained on the same examples. This comparison can be extended in future work with a transformer trained from scratch at each N , which would separate the contribution of WaM pretraining from that of the model architecture.

- **Ridge(all)**: Ridge trained on the entire training split of the target game, serving as a full-data benchmark for a non-transfer approach. Comparing against Ridge(all) shows how close the fine-tuned transformer comes to a simple model with access to substantially more target-game data.

4.4 Interpretability: Integrated Gradients

4.4.1 Why Integrated Gradients

A system meant to inform real-time decisions on a live platform also needs to explain which behavioral signals drive its predictions. If the model is responding to hit rate and momentum, those are signals the platform can monitor and intervene on. If it is responding to noise in the reaction time data, those predictions should not be trusted.

A natural first instinct for transformer interpretability is to look at attention weights, since the attention mechanism gives the model its ability to relate trials to each other. However, attention weights do not reliably measure feature importance. They describe relationships between trial positions, while our analysis requires the contribution of each input feature to the final output. As Jain and Wallace show, high attention on a token does not imply that token is causally responsible for the prediction, and different attention patterns can produce identical outputs (Jain and Wallace, 2019). Our analysis therefore requires a feature-attribution method that connects each input feature directly to the score prediction.

Integrated Gradients (IG) (Sundararajan et al., 2017) provides this feature-level explanation. IG assigns each input feature a score that reflects how much it contributed to the model’s prediction relative to a reference baseline, which in our case is the mean training session for each game. The method satisfies two important properties: completeness, meaning all the feature scores add up to the total difference between the prediction on the real input and on the baseline, and sensitivity, meaning any feature that changed the output gets a non-zero score. These properties give IG a principled basis for measuring feature importance.

4.4.2 Method

The method requires a reference baseline $\bar{\mathbf{x}}$ representing a neutral or uninformative input. We use the mean training session for each game, calculated as the average value of each feature at each trial position across all training sequences. This is preferable to a zero baseline because a zero-valued input lies outside the natural distribution of the data and would produce attributions relative to an unrealistic reference point. With the mean training session as the baseline, the attributions instead measure how each feature value differs from that of a typical session.

Given the baseline $\bar{\mathbf{x}}$ and a target input $\mathbf{x}$, Integrated Gradients constructs a straight-line path between them and accumulates the model gradients at 300 evenly spaced points along this path. The attribution of input element i , where i indexes a trial-position–feature pair in the input matrix, is approximated as

$$\text{IG}_i(\mathbf{x}) = (x_i - \bar{x}_i) \frac{1}{300} \sum_{k=0}^{299} \left. \frac{\partial F(\mathbf{z})}{\partial z_i} \right|_{\mathbf{z}=\bar{\mathbf{x}} + \frac{k}{299}(\mathbf{x}-\bar{\mathbf{x}})}. \quad (1)$$

This procedure produces an attribution matrix of shape $K \times 6$ for each session, covering every trial-position–feature pair. We apply it to 500 held-out test sessions per game across eight models: the WaM base model and seven fine-tuned game-specific models.

As a sanity check, we evaluate the completeness property for the first test session of each game. Under this property, the sum of the attributions should approximate the difference between the prediction for the observed session and the prediction for the baseline session. The resulting discrepancies are small in absolute score units, indicating that the 300-step approximation is numerically stable.

5 Results and discussion

5.1 Early session score prediction

5.1.1 Performance across trial cutoffs

At $K = 5$, the transformer achieves an RMSE of 21.24 and explains 53.7% of the variance in final session scores. At $K = 40$, the RMSE falls to 13.93 and explained variance reaches 78.2%. Table 3 presents the complete results and Figure 4 shows the performance curves across all cutoffs.

K	Ridge		SessionTransformer	
	RMSE	R^2	RMSE	R^2
5	22.50	0.480	21.24	0.537
10	20.63	0.563	18.72	0.640
20	18.03	0.666	15.63	0.749
40	16.31	0.701	13.93	0.782

Table 3: K -sweep results for WaM: RMSE and R^2 values for Ridge regression and the SessionTransformer on the held-out test set.

Ridge follows the same pattern but consistently lags behind the transformer at every value of K . Because eligibility depends on sequence length, the values at different cutoffs are based on slightly different cohorts. The results show the performance available at each cutoff in the observed data, while a strict same-cohort comparison would require restricting every experiment to sessions containing at least 40 trials.

These results show that useful predictive information is already present within the first five WaM trials, when the model explains more than half of the variance in the final score. This provides a promising basis for early monitoring, with prospective validation as the next step toward determining suitable intervention thresholds.

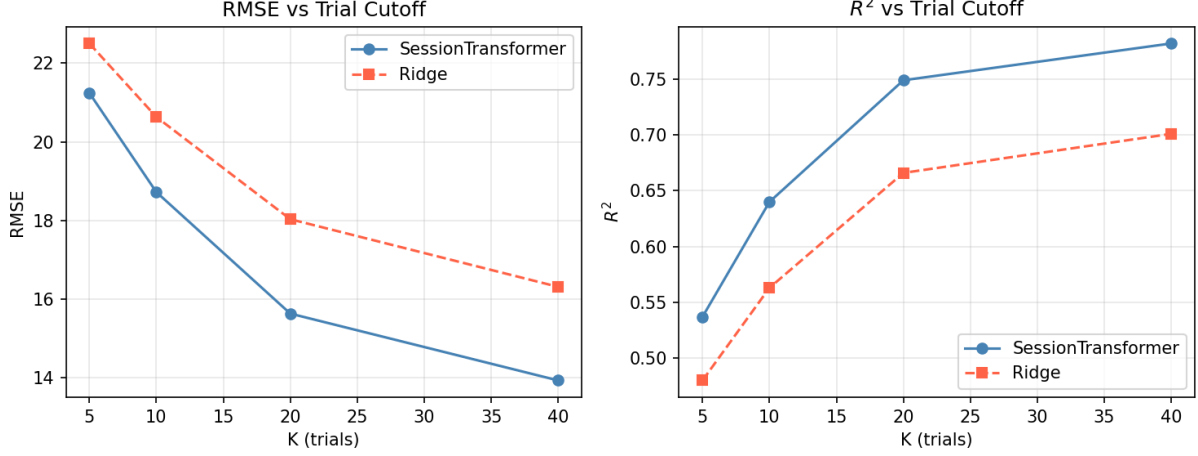


Figure 4: RMSE and R^2 versus trial cutoff K for Ridge regression and the SessionTransformer on the WaM test set. The transformer outperforms Ridge at every cutoff and the gap widens as K increases.

5.1.2 The transformer outperforms Ridge at all K

At $K = 40$, the SessionTransformer achieves an R^2 of 0.782 against 0.701 for Ridge, a 14.6% reduction in RMSE that is statistically significant ($t = 10.931$, $p < 0.001$). The advantage holds at every tested cutoff and grows with K : the transformer’s RMSE gain over Ridge rises from 5.6% at $K = 5$ to 14.6% at $K = 40$, suggesting that the model makes better use of additional trials than the aggregate baseline does.

A likely explanation for this performance difference is the models’ treatment of temporal structure. Ridge represents each session using the mean and standard deviation of each of the six features across the K trials, reducing the sequence to 12 aggregate values. Although features such as running accuracy and streak contain some temporal information, the aggregation step discards their precise positions within the sequence. Consequently, Ridge cannot distinguish between input matrices that have the same feature-wise means and standard deviations but different trial orderings.

The transformer instead processes the complete ordered sequence. It can therefore learn whether the predictive value of an observation depends on its position or on its relationship with preceding trials. The results are consistent with the presence of useful temporal information beyond that retained by the aggregate statistics, as shown in Figure 5, although they do not by themselves establish which specific temporal patterns the transformer learns.

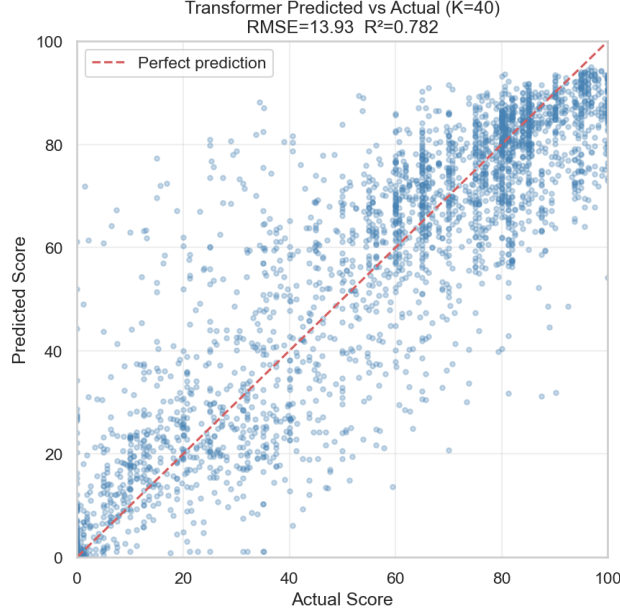


Figure 5: Predicted versus actual session scores produced by the SessionTransformer on the WaM test set at $K = 40$. The model achieves an RMSE of 13.93 and an R^2 of 0.782.

The 14.6% reduction in RMSE matters beyond model comparison because it improves the precision of early-session score estimates.

5.1.3 Training dynamics

The best checkpoint is saved at epoch 147; training stops at epoch 167. Figure 6 shows training and validation loss tracking closely throughout. There is no sign of severe overfitting, which is necessary for the model’s predictions to generalize to unseen sessions and supports the hypothesis that the learned representations capture genuine behavioral patterns rather than training-set artifacts.

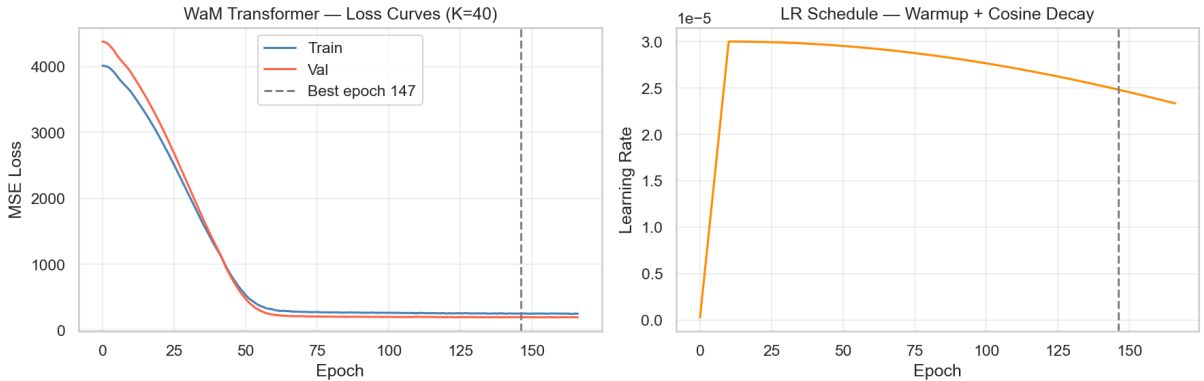


Figure 6: Training and validation MSE loss curves for the WaM base model ($K = 40$), with the learning rate schedule on the right. Best checkpoint at epoch 147, early stop at epoch 167.

5.2 Transfer learning across games

5.2.1 Zero-shot transfer fails

Applying the WaM model directly to any target game produces near-zero or negative R^2 on all but one game. React achieves $R^2 = 0.134$ zero-shot, owing to its structural similarity to WaM: both games use $K = 40$ and share an accuracy-based trial format. That React alone yields a positive zero-shot result suggests that some behavioral representations do transfer without adaptation, but the output head, which is calibrated to WaM’s score distribution, acts as a bottleneck when applied to games with different score ranges and dynamics. The practical takeaway is that a small amount of fine-tuning is required, but the question is how little is enough.

5.2.2 Fine-tuning with limited target data

Figure 7 shows the R^2 versus N curves. At $N = 20$, the fine-tuned transformer outperforms Ridge trained on the same 20 sessions in Assembly, Maestro, Odd Egg, React, and Stacker. Magic Deck shows an advantage at $N = 10$, while Shuffled first outperforms its same-budget Ridge comparison at $N = 100$, though both models remain below zero at that budget. None of the fine-tuned models matches Ridge(all) at $N = 20$.

These results show that WaM pretraining can provide an advantage when target-game data is limited, although the size of that advantage varies by game. The strongest absolute performance is reached at $N = 200$ or $N = 500$, and the low-budget points should be read as single-run estimates because each budget uses one independently sampled training set. Games with very short early windows, such as Shuffled ($K = 5$), require more data before positive predictive performance emerges.

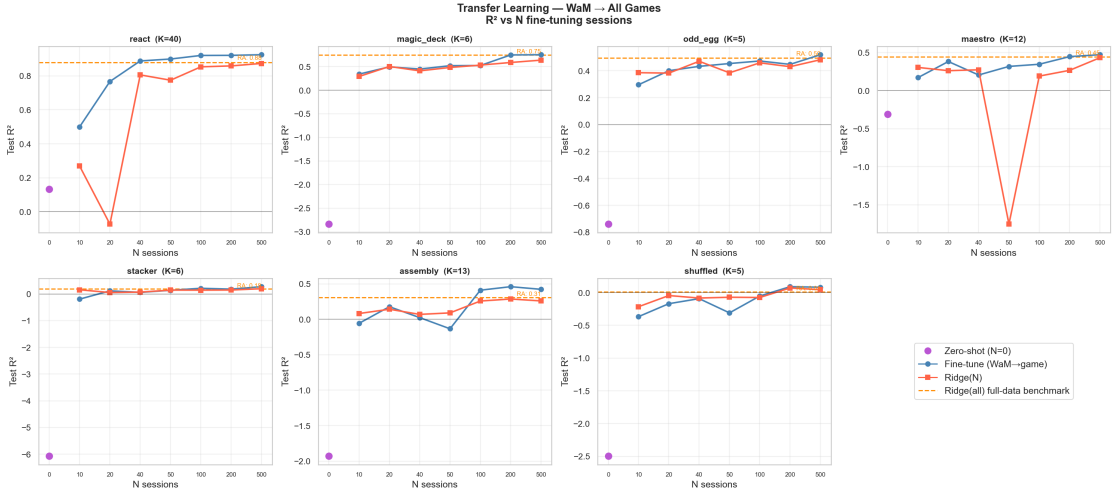


Figure 7: R^2 as a function of fine-tuning budget N for each game. Fine-tune (WaM $\rightarrow$ game) is shown in blue, Ridge(N) in red, Ridge(all) as a dashed orange line, and the zero-shot result as a purple dot.

5.2.3 Transfer performance varies by game

The results split into three tiers (Table 4):

Strong transfer ($R^2 > 0.7$): React (0.925) and Magic Deck (0.754). React is the clearest success. It shares WaM’s $K = 40$ window and accuracy-based trial format, and the fine-tuned transformer exceeds the Ridge(all) benchmark from $N = 40$ onward. Magic Deck reaches comparable performance to Ridge(all) at $N = 500$ despite its much shorter sequence.

Moderate transfer ($R^2 = 0.45\text{--}0.52$): Odd Egg (0.520), Assembly (0.459), and Maestro (0.473). These games have shorter early windows ($K = 5\text{--}13$) and different game mechanics. Transfer works but leaves more room for improvement than the high- K games. Note that Maestro’s improvement over Ridge is not statistically significant ($t = 1.801$, $p = 0.072$), indicating that the pretrained representations provide no measurable advantage over Ridge for this game specifically.

Weak transfer ($R^2 < 0.3$): Stacker (0.274) and Shuffled (0.092). Both games use short early windows of five or six trials, and the Ridge(all) benchmarks are also low at 0.192 and 0.012. These values show that the evaluated feature representation contains limited early predictive signal for both model classes. Stacker still shows a statistically significant improvement over Ridge trained at the same N ($p < 0.001$), while Shuffled does not.

Game	K	Zero-shot R^2	Best N	TF R^2	Ridge (N) R^2	Ridge (all) R^2	Sig.
React	40	0.134	500	0.925	0.873	0.878	***
Magic Deck	6	−2.840	500	0.754	0.638	0.751	***
Odd Egg	5	−0.740	500	0.520	0.483	0.496	***
Maestro	12	−0.310	500	0.473	0.433	0.447	ns
Assembly	13	−1.929	200	0.459	0.286	0.307	***
Stacker	6	−6.072	500	0.274	0.197	0.192	***
Shuffled	5	−2.497	200	0.092	0.072	0.012	ns

Table 4: Transfer-learning results by game. Best N denotes the fine-tuning budget at which the transformer achieves its highest R^2 . Ridge (N) is trained on the same number of sessions, while Ridge (all) uses the complete training split. Significance is based on a paired t -test comparing the transformer with Ridge (N) at the best N : *** $p < 0.001$; ns indicates no statistically significant difference.

5.2.4 The Shuffled case

Shuffled deserves specific attention. $R^2 = 0.092$ at best $N = 200$, and the difference from Ridge is not statistically significant ($t = 1.353$, $p = 0.176$). Ridge(all) on the full Shuffled training set reaches $R^2 = 0.012$, so both evaluated approaches find little predictive signal in the first five trials under the shared feature representation.

This result shows that the useful early window needs to be evaluated separately for each game. Shuffled may require a longer prefix, features that better reflect its mechanics,

or both. The current experiment does not establish a universal minimum number of trials for reliable prediction.

5.3 Interpretability

5.3.1 Reaction time and streak are leading predictors across games

Figures 8 and 9 present the normalized absolute IG attribution of each feature across all eight models. On average, **rt_norm** is the dominant signal, accounting for 52.6% of the total attribution, while **streak** ranks second with 31.9%. Together, they account for 84.5% of the attribution. The binary **hit** feature ranks fourth with 4.9%, below **rt_rolling_std_norm** at 6.3%.

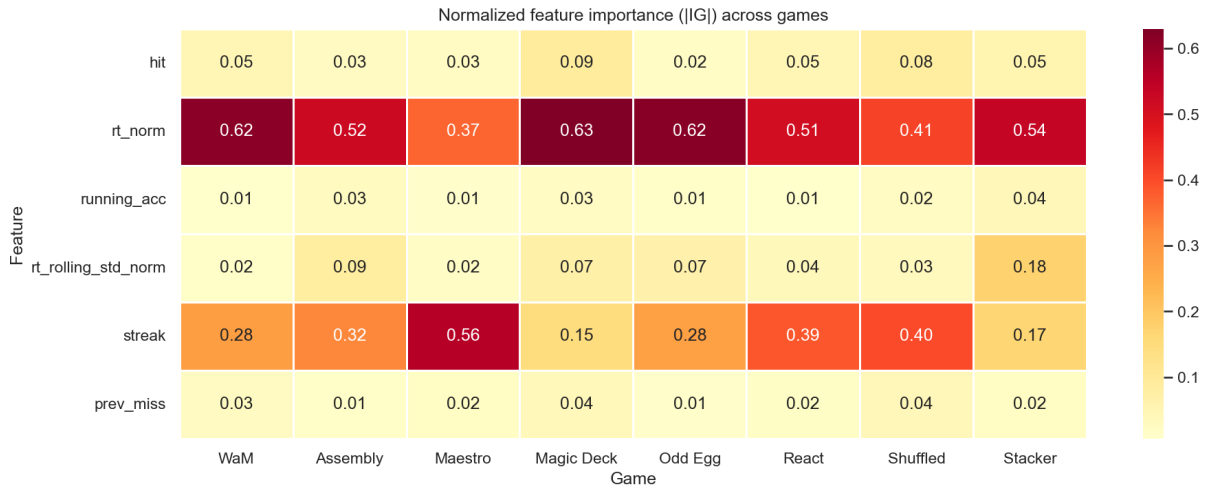


Figure 8: Normalized mean absolute Integrated Gradients attribution by feature across all eight games. Each column sums to one. Reaction time and streak collectively account for most of the attribution across games.

Although the relative importance of the features varies between games, reaction time and streak collectively dominate the cross-game results. **rt_norm** is the most important feature in seven of the eight games, while **streak** is the most important feature for Maestro. The consistency of their combined importance across games with different mechanics and score distributions is the central interpretability finding.

A plausible cognitive interpretation is that reaction time captures several aspects of momentary performance, including processing speed, attention, and motor readiness. Streak provides complementary information about sustained success or failure across consecutive trials. Psychological momentum in skilled performance is well documented (Iso-Ahola and Dotson, 2016), and the IG results indicate that the model assigns substantial importance to the explicitly engineered streak feature.

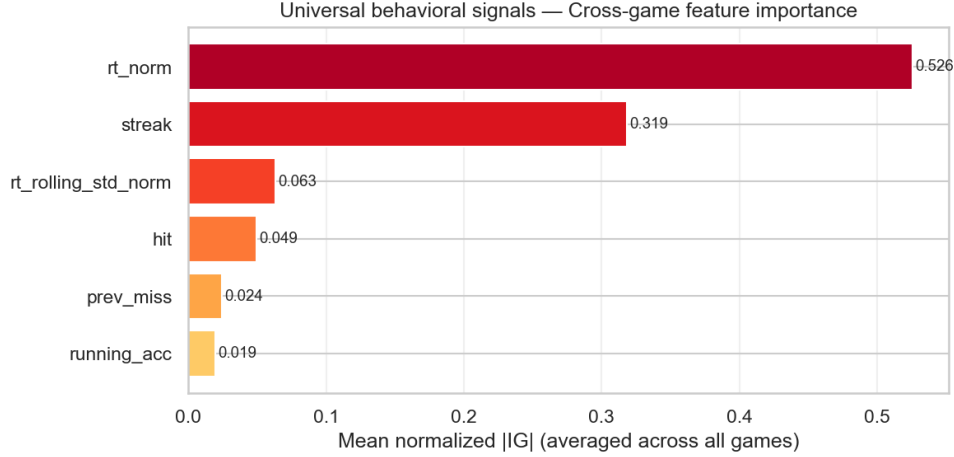


Figure 9: Cross-game mean feature importance ranked by normalized absolute attribution. **rt_norm** ranks first (0.526) and **streak** ranks second (0.319), together accounting for 84.5% of the total attribution.

These findings identify reaction time and streak as the two principal signals to monitor during a session. Changes in these features may provide useful early indications of how a session is developing. Further analysis of signed attributions would be required to determine the direction of these relationships and establish whether increasing reaction time or a broken streak specifically signals deterioration.

5.3.2 Direct accuracy measures are weaker than expected

hit and **running_acc** account for only 4.8% and 1.9% of the mean absolute attribution, respectively, despite average hit rate being strongly correlated with the final session score. Reaction time is continuous and distinguishes between responses that binary outcomes cannot: two correct trials can differ substantially in speed, and the model appears to weight that difference. The strong importance of **streak** suggests that accuracy is most informative when represented as a pattern across consecutive trials. Monitoring accuracy alone gives an incomplete picture of a session; streak and reaction time carry the signal that actually moves the prediction.

5.3.3 Temporal attribution patterns

Figure 10 shows how attribution is distributed across trial positions. Short- K games (Shuffled, Odd Egg, Magic Deck, Stacker: $K = 5-6$) do not share a common pattern. Shuffled places the greatest weight on its first trial, while Odd Egg and Magic Deck emphasize later trials. Stacker distributes attribution relatively evenly after assigning little importance to its first trial. Longer games (WaM, React: $K = 40$) distribute attribution more broadly, with React displaying a clearer recency pattern.

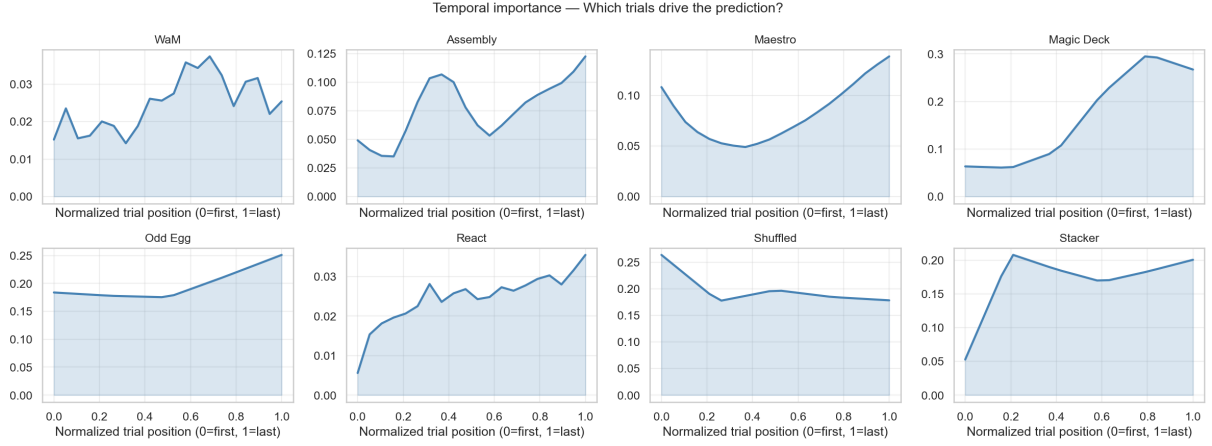


Figure 10: Normalized mean absolute attribution, summed across features, per normalized trial position for each game. Temporal importance varies substantially across games.

This suggests that intervention timing may need to be calibrated separately for each game. However, these profiles are based on predictions using the complete K -trial window and do not establish the earliest point at which a reliable intervention could occur. Determining that point would require evaluating the models on progressively shorter input prefixes.

5.3.4 Streak vs reaction time through time

Figure 11 reveals which feature drives the temporal patterns observed in Figure 10. Reaction time dominates across most trial positions in six games. Magic Deck’s late peak is driven primarily by reaction time, whereas Maestro’s late increase is driven by streak. Shuffled shows the clearest transition between the two, while both features become more influential toward the end of React.

The main finding is that similar temporal attribution patterns can arise from different behavioral signals. The cross-game averages therefore conceal important differences in how each model uses reaction time and streak over the course of a session.

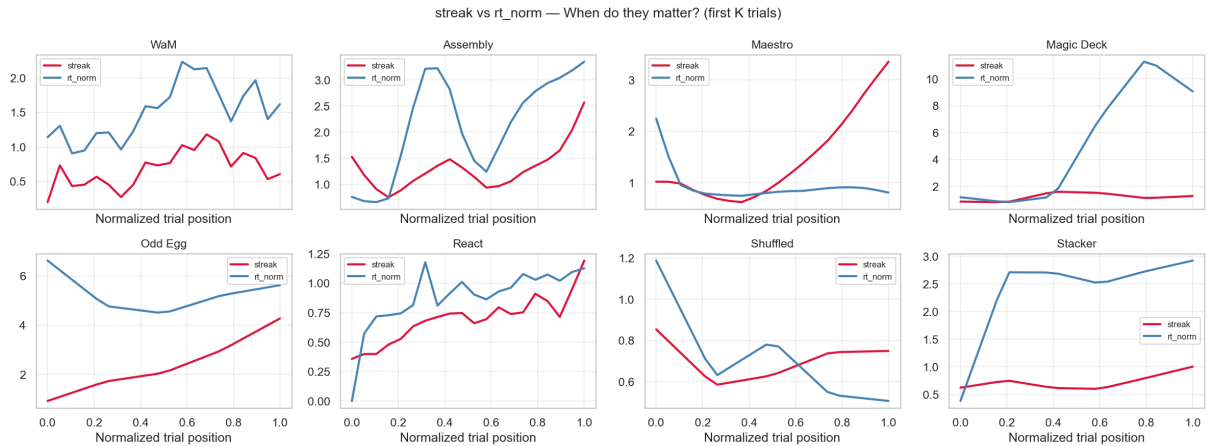


Figure 11: Temporal attribution for **streak** and **rt_norm** separately for each game.

5.4 Synthesis

The three components fit together into one system. The base model shows that session outcomes are predictable well before a session ends: 53.7% of variance explained at five trials, 78.2% at forty. Across the seven additional games studied, fine-tuning often outperforms Ridge trained on the same limited data budget, although performance varies by game and the strongest results require 200 to 500 labeled sessions. The interpretability analysis identifies reaction time and streak as the two leading predictive signals on average across the eight games.

WaM and React provide the strongest candidates for prospective validation. The harder cases may benefit from game-specific features and longer early windows. Reaction time and streak also provide a starting point for designing game-specific monitoring strategies, which would need to be tested prospectively before informing difficulty adjustment or intervention.

6 Limitations and future work

6.1 Limitations

The dataset available for this project reflects how a live platform actually grows: games launched at different times, some have far more users than others, and a few did not have enough event-level data to be included in the analysis at all. This affected the scope of what we could test, but it does not change what the results show for the games we did have data for. The limitations below concern the feature representation, experimental design, and model architecture.

The most consequential of those decisions is the shared feature set. To build a model that works across all eight games, we needed a single feature representation that could be computed from any game’s event data. For WaM and React, where each trial produces a clear hit-or-miss outcome and a reaction time, that mapping is straightforward and loses very little. For games like *Stacker*, *Assembly*, and *Maestro*, the raw behavioral data is structured differently: move optimality scores, gear selection rankings, pattern similarity judgments. Translating those into the same six features captures something real but inevitably leaves out signals that are specific to how each of those games works. This is the most likely explanation for why transfer performance varies so much across the game catalog, and why *Shuffled*, which differs most from WaM in both mechanics and cognitive demands, ends up being the hardest case.

Sequence length is a separate but related problem. Several games in the dataset operate at $K = 5$ or $K = 6$, compared to $K = 40$ for WaM and React. A transformer with five tokens does not have much to work with. Self-attention learns from relationships between positions in the sequence, and when there are barely any positions, that mechanism cannot do a lot regardless of how well the pretrained representations travel across games. The weaker results for short-window games come down to this as much as anything else.

The few-shot experiments use one independently sampled training set at each value of N . This keeps the experimental procedure simple, but it also means that differences between adjacent budgets reflect both sample size and the particular sessions drawn. Repeating the experiment across several seeds and reporting uncertainty intervals would provide a more reliable estimate of data efficiency. The WaM K -sweep has a related cohort difference because each cutoff includes all sessions long enough to qualify. A controlled estimate of the effect of K would use the same sessions at every cutoff.

The model uses absolute sinusoidal positional encoding, which assigns a fixed representation to each trial position. The current experiments do not establish whether this choice limits transfer across games with different values of K , since each target model is fine-tuned separately. A useful extension would be to compare it with relative positional encoding, which represents the distance between trial positions directly (López-Ávila et al., 2024). The model also applies mean pooling to the encoder outputs. Attention can already place position-dependent information into those outputs, so mean pooling does not imply that every trial has equal influence on the prediction. However, comparing it with a learned pooling mechanism would test whether explicitly weighting the final trial representations improves performance.

Integrated Gradients interpolates between the observed input and a reference baseline along a straight path. The trial positions remain fixed, but the interpolated feature values may form combinations that do not resemble observed sessions or preserve dependencies between consecutive trials. The resulting rankings should therefore be interpreted as the model’s sensitivity to deviations from a typical session, without assigning those relationships a causal meaning.

6.2 Future work

One useful extension is to develop game-specific features that reflect what each game measures. For games such as Stacker and Assembly, the native behavioral signals include move efficiency and selection accuracy, which are only partly represented by the shared hit-or-miss format. Features designed around these mechanics could improve performance in games where the current representation fits poorly.

A second extension is joint pretraining across several data-rich games. This would expose the model to a wider range of trial structures and behavioral patterns before adaptation to a target game. The resulting model could then be compared with WaM-only pretraining to determine whether broader source data improves transfer to games with different mechanics.

The architecture could be extended by comparing absolute and relative positional encoding and by evaluating learned pooling in place of global mean pooling. These experiments would show whether representing pairwise trial distances or explicitly weighting the encoder outputs improves performance across games with different sequence lengths.

The interpretability analysis could also compare standard Integrated Gradients with a temporality-aware method such as TIMING (Jang et al., 2025). This would test whether

accounting for temporal relationships and reducing unrealistic intermediate inputs changes the feature rankings.

The longer-term opportunity for Virtuleap goes beyond within-session prediction. Once more session data builds up over time across users, cross-session modeling becomes possible: tracking how a user’s performance evolves across many sessions over weeks or months, and identifying early signs of disengagement or cognitive change before they become visible in aggregate statistics. The architecture built here is a natural starting point for that, and the interpretability findings about which signals matter most give a concrete basis for deciding what to monitor at the cross-session level.

7 Conclusion

Virtuleap collects detailed trial-by-trial behavioral data from every game session: whether a user hit or missed, how fast they responded, and how their performance evolved across the session. This thesis shows that these data contain predictive information well before the end of a session. It also shows that a model pretrained on WaM can be adapted to seven additional games, with the value of transfer varying across games and data budgets.

On Whack-a-Mole, the SessionTransformer explains 78.2% of the variance in final session scores from just the first 40 trials, a statistically significant improvement over Ridge regression ($R^2 = 0.782$ vs 0.701 , $p < 0.001$). At five trials, it already explains 53.7% of the variance. This improvement suggests that the model captures how performance develops across a session through the sequence of hits and misses, the consistency of reaction times, and patterns of momentum. The resulting early indication of where a session is heading could help flag a struggling user, tighten the difficulty curve, or deliver feedback while it can still change the outcome.

The transfer results show this capability extends across the catalog. The WaM-pretrained model outperforms Ridge at the same data budget from as few as $N = 20$ sessions in most games. React reaches $R^2 = 0.925$. Pretraining reduces the amount of game-specific data needed to begin adaptation because the model already has a learned representation of trial sequences and can focus on how those sequences connect to a new game’s score distribution. The strongest results need 200 to 500 fine-tuning sessions. Shuffled is the hardest case because five trials give the model little information, and its mechanics are further from WaM than any other game in the set. This provides a useful design insight: the prediction system is most reliable for games with longer early windows, and very short windows limit what is possible regardless of the approach.

The interpretability results are the most directly useful for Virtuleap. Reaction time and streak drive most of the predictions across all eight games. Hit rate and running accuracy matter less than their correlation with final scores would suggest. The model is reading how a session develops: whether responses are getting faster or slower, whether the user’s run of successes is holding or breaking down. This also shifts across games, with reaction time dominating in Magic Deck and streak doing more work in Maestro. These differences call for separate monitoring thresholds and intervention triggers for each game.

Taken together, the results demonstrate that partial-session behavior contains useful predictive information across a varied set of VR cognitive games. Performance varies by game, which points to clear opportunities for improvement through game-specific features and additional target-game data. Prospective validation can now build on this foundation by establishing suitable thresholds for each game and testing whether the same signals can track changes within users across weeks and months of training.

Appendices

A Dataset summary

Table A.1 reports the number of completed, non-tutorial sessions and unique users per game after matching sessions to their event records, together with the mean and standard deviation of session scores. WaM is the largest game by session count and the only one used for pretraining.

Game	Sessions	Users	Mean score	Std score
WaM	23,345	4,932	54.8	31.1
React	9,986	1,227	51.5	31.2
Assembly	16,225	3,227	59.4	24.5
Maestro	12,129	1,889	56.1	24.4
Magic Deck	13,233	2,478	66.6	25.1
Odd Egg	15,176	3,140	65.0	31.8
Stacker	11,580	2,346	64.6	20.9
Shuffled	8,241	1,822	69.1	22.6
Total	109,915	<i>n/a</i>		

Table A.1: Per-game dataset statistics after filtering to completed, non-tutorial sessions with a recorded score. The total reports the sum of sessions across games. A total user count is not reported because the same user may appear in multiple games.

B Transfer learning results

Table B.1 reports R^2 for the fine-tuned transformer and Ridge regression at four selected fine-tuning budgets, for all seven target games. The complete data-efficiency curves, including $N \in \{40, 50, 200\}$, are shown in Figure 7. Bold entries indicate that the transformer outperforms Ridge at that budget.

	$N = 10$		$N = 20$		$N = 100$		$N = 500$	
Game	TF	Ridge	TF	Ridge	TF	Ridge	TF	Ridge
React	0.500	0.269	0.767	-0.073	0.920	0.853	0.925	0.873
Magic Deck	0.342	0.292	0.496	0.504	0.524	0.533	0.754	0.638
Odd Egg	0.296	0.386	0.401	0.383	0.473	0.459	0.520	0.483
Maestro	0.172	0.306	0.383	0.263	0.348	0.192	0.473	0.433
Assembly	-0.057	0.081	0.177	0.142	0.410	0.257	0.422*	0.259
Stacker	-0.193	0.157	0.118	0.055	0.213	0.143	0.274	0.197
Shuffled	-0.366	-0.214	-0.168	-0.044	-0.051*	-0.074	0.083	0.046

Table B.1: R^2 for the fine-tuned transformer (TF) and Ridge at selected budgets N . Bold marks the better model at each budget. Assembly at $N = 500$ is marked * because TF performance is non-monotonic (lower than at $N = 200$), making this estimate noisy. Shuffled at $N = 100$ is marked * because both models are near zero and the gap is not meaningful. The table shows four selected budgets; $N \in \{40, 50, 200\}$ are omitted here but included in Figure 7.

C Pairwise feature correlations

Figure C.1 shows the pairwise Pearson correlations between the six input features on the WaM training data. `running_acc` and `streak` are the most strongly correlated pair ($r = 0.62$), reflecting that both track accuracy over consecutive trials. `hit` correlates positively with `running_acc` ($r = 0.57$) and `streak` ($r = 0.55$), and negatively with `prev_miss` ($r = -0.41$). `rt_norm` shows weak correlations with most features, suggesting it carries information largely independent of the accuracy-based signals.

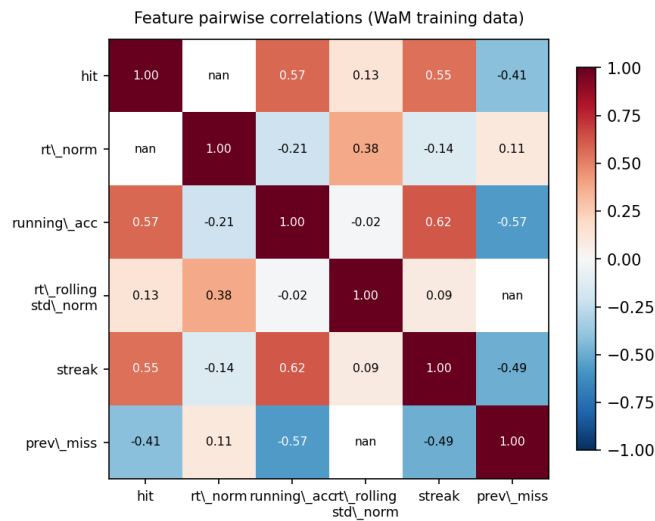


Figure C.1: Pairwise Pearson correlations between the six input features on the WaM training data. Each cell reports the correlation between the row and column feature; `nan` entries indicate pairs where one feature had zero variance.

D WaM performance across sessions

Figure D.1 shows the mean and median WaM score over the first ten session positions among users who completed at least three sessions. Scores rise slightly over the first three sessions and then decline across the later positions. The number of contributing users falls as the session number increases, so this pattern is descriptive and should not be interpreted as a within-user learning trajectory.

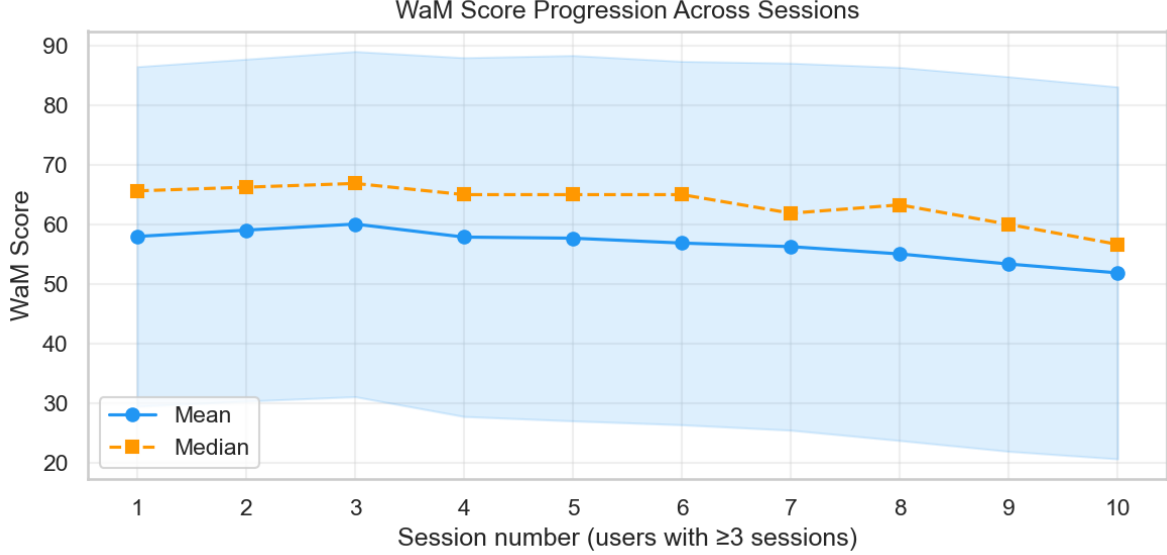


Figure D.1: Mean and median WaM score over the first ten session positions for users with at least three sessions. The shaded band shows the mean plus or minus one standard deviation.

E Ridge regression baseline

The Ridge regression baseline used throughout this paper represents each session as a fixed-length feature vector by aggregating the $K \times 6$ trial matrix into 12 summary statistics: the mean and standard deviation of each of the six features (`hit`, `rt_norm`, `running_acc`, `rt_rolling_std_norm`, `streak`, `prev_miss`) computed across the K trials. This aggregation discards all information about the ordering of trials within the session.

The Ridge regularization strength is fixed at $\alpha = 1.0$ across all experiments. The same normalization parameters used to construct the transformer input features are applied to the Ridge features, ensuring the baseline and the transformer see equivalent inputs at the feature level. The key difference is that Ridge sees only the 12 aggregate statistics while the transformer processes the full ordered sequence.

F Model architecture and hyperparameters

Hyperparameter	Value
<i>Architecture</i>	
Model dimension d	64
Encoder layers	2
Attention heads	8
FFN width	128
Dropout	0.1
Total parameters	69,505
<i>Pretraining (WaM)</i>	
Optimizer	AdamW
Learning rate	3×10^{-5}
Weight decay	10^{-4}
Batch size	128
Max epochs	500
Warmup epochs	10
LR schedule	Cosine annealing $\rightarrow 10^{-6}$
Early stopping patience	20 epochs
Gradient clip norm	1.0
<i>Fine-tuning (target games)</i>	
Learning rate	10^{-4}
All other settings	As above

Table F.1: SessionTransformer architecture and training hyperparameters.

References

- Federal Trade Commission (2016). Lumosity to pay \$2 million to settle FTC deceptive advertising charges.
- Iso-Ahola, S. E. and Dotson, C. O. (2016). Psychological momentum: A key to continued success. *Frontiers in Psychology*.
- Jain, S. and Wallace, B. C. (2019). Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 3543–3556.
- Jang, H., Kim, C., and Yang, E. (2025). TIMING: Temporally-aware integrated gradients for time series. In *Proceedings of the 42nd International Conference on Machine Learning*.
- Jutten, R. J., Amariglio, R. E., Maruff, P., Properzi, M. J., Rentz, D. M., Johnson, K. A., Sperling, R. A., and Papp, K. V. (2023). Increased intraindividual variability in reaction time is associated with emerging cognitive decline. *PMC*.
- Li, G., Liu, Y., and Chen, A. (2026). Working memory training: A meta-analysis. *npj Digital Medicine*.
- Lin, X., Ye, M., Gong, Y., Buracas, G., Basiou, N., Divakaran, A., and Yao, Y. (2021). Cross-domain few-shot learning: Fine-tuning vs. metric learning.
- López-Ávila, A., Du, J., Shimary, A., and Li, Z. (2024). Relative positional encoding for transformers.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*.
- Pandey, S. and Karypis, G. (2019). A self-attentive model for knowledge tracing. In *Proceedings of the 12th International Conference on Educational Data Mining*.
- Pang, Y., Singh, A., Chakraborty, S., Charness, N., Boot, W. R., and He, Z. (2024). Predicting adherence in gamified cognitive training from early performance metrics. *PLOS ONE*.
- Piech, C., Spencer, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., and Sohl-Dickstein, J. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems*, 28.
- Saredakis, D. et al. (2022). Enhance VR: A multisensory approach to cognitive training and monitoring. *Frontiers in Digital Health*, 4.
- Shin, D., Shim, Y., Yu, H., Lee, S., Kim, B., and Choi, Y. (2021). SAINT+: Integrating temporal features for EdNet correctness prediction. In *Proceedings of the 11th International Learning Analytics and Knowledge Conference*, pages 490–496.

- Simons, D. J. et al. (2019). Near and far transfer in cognitive training: A reanalysis. *Collabra: Psychology*, 5(1):18.
- Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 3319–3328.
- Wisner, P., Soares, S. A., Einarson, K. A., and Sommer, M. O. A. (2022). Predicting expert vs. novice performance from VR behavioral signals. *PMC*.
- Zhao, Q. et al. (2023). Few-shot fine-tuning is all you need for domain adaptation.